

Veridion Build Prompts - Organized by Priority Flow

Veridion — Build Prompts, Organized by Priority Flow

PART 0 — All API keys / credentials, collected once up front

FLOW 1 — The Core Lifecycle, No AI Yet

FLOW 2 — The Core AI Decision (OCR, Matching, Compliance, the Context Gate)

FLOW 3 — Fraud Detection and Vendor Risk (completing the Decision Engine)

FLOW 4 — Quotation Ranking and Vendor Communication (completing all 8 agents)

FLOW 5 — Multi-Company Isolation and Legacy Vendor Import

FLOW 6 — Vendor Voice Notification (OmniDimension)

FLOW 7 — Admin Interface, Testing Data, and Evaluation

FLOW 8 — Deployment

If time runs out, this is the honest cutoff order

Veridion — Build Prompts, Organized by Priority Flow

Complete each flow fully before starting the next. Each flow ends in something you can actually demo.

Why this order: the single biggest risk for a team your size is having eight half-finished pieces instead of five fully-working ones. Each flow below is a complete, demoable increment — if you run out of time after Flow 4, you still have a working, presentable project. Don't start Flow 2 until Flow 1 is fully working end-to-end with real data in the database.

Stack: HTML/CSS/Bootstrap 5 + vanilla JS (frontend), Node.js + Express (backend), PostgreSQL, RabbitMQ, Neo4j, Groq (AI), OmniDimension (voice notifications), Docker + Render (deployment).

PART 0 — All API keys / credentials, collected once up front

Key	Where to get it	Needed from which flow
<code>DATABASE_URL</code>	Local Docker Postgres, or Render Postgres	Flow 1
<code>JWT_SECRET</code>	Generate yourself: <code>openssl rand -base64 32</code>	Flow 1
<code>GROQ_API_KEY</code>	console.groq.com — free tier	Flow 2
<code>RABBITMQ_URL</code>	Local Docker, or CloudAMQP free tier	Flow 2
<code>NEO4J_URI</code> , <code>NEO4J_USER</code> , <code>NEO4J_PASSWORD</code>	Neo4j AuraDB free tier, or local Docker	Flow 3
<code>OMNIDIMENSION_API_KEY</code> , <code>OMNIDIMENSION_AGENT_ID</code> , <code>OMNIDIMENSION_OUTBOUND_NUMBER_ID</code>	omnidim.io dashboard, free to start	Flow 6
<code>GSTN_SANDBOX_API_KEY</code> (optional)	A GSP sandbox provider — skip and use the mock function if unavailable	Flow 2

FLOW 1 — The Core Lifecycle, No AI Yet

Milestone at the end: you can register a vendor and a company, post a requirement, get a quotation, manually accept it, record a delivery, and upload an invoice — all through real pages, hitting a real database. Nothing AI-related exists yet, and that’s correct at this stage.

This is the single most important flow. Everything else is worthless if this isn’t solid, and everything else becomes much easier once this works, because it defines the real data every AI agent will operate on.

Prompt 1.1 — Repository structure

```
Create a monorepo folder structure for a project called Veridion with these top-level folders:
- /backend (Node.js + Express API)
- /ai-service (Node.js service for AI agents — empty for now, filled in Flow 2 onward)
- /frontend/vendor (HTML/CSS/Bootstrap pages for the Vendor interface)
- /frontend/company (HTML/CSS/Bootstrap pages for the Company Profile interface — shared by Company Admin, Procurement, Finance, Warehouse roles)
- /frontend/admin (HTML/CSS/Bootstrap pages for the Platform Admin interface)
- /frontend/shared (shared CSS, a Bootstrap navbar partial, and a shared JS file for API calls and JWT handling)
- /db (SQL migration files)
- /docker (Dockerfiles and docker-compose.yml)

Set up package.json in /backend with Express, pg, jsonwebtoken, bcrypt, multer, pdfkit, dotenv, and cors as dependencies. Add a root README explaining the folder structure.
```

Prompt 1.2 — Docker Compose for local development (Postgres only for now)

```
Create a docker-compose.yml in /docker that spins up:
- postgres:16 with a named volume, exposing 5432, POSTGRES_DB=veridion
- the backend service (built from /backend/Dockerfile), exposing 4000, depending on postgres

Add RabbitMQ and Neo4j services to this same file but commented out for now — we will uncomment them in Flow 2 and Flow 3. Create a simple Dockerfile for /backend using node:20-alpine. Add a .env.example listing DATABASE_URL and JWT_SECRET.
```

Prompt 1.3 — Core schema migration

```
Write a PostgreSQL migration file (001_core_schema.sql) creating these tables, with proper primary keys, foreign keys, and sensible NOT NULL constraints:

users (id, name, email UNIQUE, password_hash, role, company_id FK nullable, created_at)
-- role must be one of: 'vendor', 'company_admin', 'procurement', 'finance', 'warehouse', 'platform_admin'
companies (id, name, gstin, address, industry_type, created_by FK to users, created_at)
vendors (id, user_id FK, company_name, gstin, pan, business_reg_proof_path, bank_account_number, bank_ifsc, verification_status DEFAULT 'pending', trust_score, phone_number, preferred_notification_channel DEFAULT 'sms', created_at)
invitations (id, company_id FK, invited_email, invited_role, invited_by FK to users, status DEFAULT 'pending', token, created_at, accepted_at)
requirements (id, company_id FK, title, description, category, quantity, unit, deadline, status DEFAULT 'open', created_by FK to users, created_at)
quotations (id, requirement_id FK, vendor_id FK, price, delivery_days, notes, ai_rank_score, ai_rank_reasoning, status DEFAULT 'submitted', submitted_at)
purchase_orders (id, requirement_id FK, quotation_id FK, vendor_id FK, company_id FK, agreed_price, agreed_quantity, agreed_delivery_date, status DEFAULT 'issued', po_document_path, fulfillment_status DEFAULT 'pending', cumulative_invoiced_quantity DEFAULT 0, cumulative_invoiced_amount DEFAULT 0, created_at)
goods_receipt_notes (id, po_id FK, received_quantity, received_date, warehouse_notes, recorded_by FK to users, discrepancy_flag DEFAULT false, expected_next_delivery_date, next_delivery_notes)
invoices (id, po_id FK, grn_id FK nullable, vendor_id FK, invoice_number, invoice_amount, gst_amount, gstin_on_invoice, invoice_file_path, status DEFAULT 'submitted', due_date, submitted_at)

Use SERIAL primary keys. Add indexes on all foreign key columns. Then write a seed script (seed.sql) inserting
```

```
1 platform_admin, 2 companies each with a company_admin/procurement/finance/warehouse user, and 3 vendors
in
different verification_status states, so every role can log in immediately after migration.
```

Prompt 1.4 — Auth and RBAC

```
Build Express routes and middleware using the users/vendors/companies/invitations tables:
- POST /api/auth/register/vendor — collects company_name, gstin, pan, phone_number, uploads
business_reg_proof
  via multer, creates a users row (role='vendor') and a vendors row (verification_status='pending')
- POST /api/auth/register/company — creates a companies row and a users row with role='company_admin'
- POST /api/auth/login — validates with bcrypt, returns a JWT with user id, role, company_id
- requireAuth middleware verifying the JWT
- requireRole(...roles) middleware factory returning 403 if req.user.role isn't allowed
- POST /api/company/invite (company_admin only) — creates an invitations row with a random token
- POST /api/auth/accept-invite/:token — sets a password, creates the users row with the invited
role/company_id
```

Prompt 1.5 — Core lifecycle routes (no AI — GRN discrepancy detection only)

```
Build Express routes, each protected by requireAuth + requireRole:
- POST /api/requirements (procurement)
- GET /api/requirements (vendor) — filterable by category
- POST /api/quotations (vendor)
- GET /api/requirements/:id/quotations (procurement) — for now, just return them sorted by price; AI
ranking
  is added in Flow 4
- POST /api/quotations/:id/accept (procurement) — inside a transaction: sets quotations.status='selected',
  creates a purchase_orders row, generates a PDF with pdfkit, sets po_document_path
- POST /api/grn (warehouse) — creates a goods_receipt_notes row; if received_quantity < the PO's
  agreed_quantity, sets discrepancy_flag=true and fulfillment_status='partially_fulfilled', else
  'fulfilled';
  accept optional expected_next_delivery_date/next_delivery_notes only when a shortfall occurs
- GET /api/purchase-orders/:id — for the warehouse role, strip agreed_price and all financial fields from
  the
  response before sending; every other role gets the full record
- POST /api/invoices (vendor) — uploads a file via multer, creates an invoices row with status='submitted'
  (for now, nothing further happens to it automatically — AI processing is wired up starting in Flow 2)
```

Prompt 1.6 — Frontend: shared layout and auth

```
In /frontend/shared, create:
- style.css with a clean navy/amber color scheme
- navbar.js rendering a Bootstrap 5 navbar based on the JWT's decoded role, showing only relevant links
- api.js with fetchWithAuth(url, options) attaching the JWT as a Bearer token, redirecting to /login.html
  on 401
- login.html posting to /api/auth/login, storing the JWT, redirecting by role to the right dashboard
```

Prompt 1.7 — Frontend: the core pages for each role

```
Build these Bootstrap pages, each calling the routes from Prompt 1.5 via api.js:

/frontend/vendor: register.html, dashboard.html, requirements.html (browse + submit quotation modal),
  my-quotations.html, my-invoices.html (with an Upload Invoice modal)

/frontend/company: dashboard.html, team.html (company_admin — invite modal), post-requirement.html
  (procurement),
  quotation-comparison.html (procurement — plain table sorted by price for now), grn-entry.html (warehouse
  —
  PO lookup with item/quantity/date only, no price fields anywhere in the HTML or API calls)

Use Bootstrap badges for status everywhere (grey=submitted, blue=selected/issued, green=fulfilled,
  amber=partially_fulfilled).
```

Stop here and test thoroughly before moving on. Walk through the entire flow manually: register a vendor, register a company, post a requirement, submit a quotation, accept it, record a GRN (try both a full and a partial delivery), upload an invoice. If any of this is shaky, fix it now — every later flow builds directly on top of this data.

FLOW 2 — The Core AI Decision (OCR, Matching, Compliance,

the Context Gate)

Milestone at the end: an uploaded invoice gets automatically read, checked, and either auto-approved or flagged, with a visible confidence-weighted reasoning shown on the Invoice Review page. This is your project's actual novel contribution — get this fully working before anything else AI-related.

Prompt 2.1 — AI schema migration

```
Write a migration file (002_ai_schema.sql) creating:
document_extractions (id, invoice_id FK, raw_ocr_text, structured_data JSONB, confidence_score,
bounding_boxes JSONB)
compliance_checks (id, invoice_id FK, gst_valid, gst_details JSONB, issues_found TEXT[], confidence_score,
data_volume, gst_rate_used, gst_rate_reference_date)
matching_results (id, invoice_id FK, po_match_score, grn_match_score, mismatch_fields JSONB, overall_match,
confidence_score, data_volume, issue_type DEFAULT 'mismatch')
-- issue_type: 'informational' / 'mismatch' / 'fraud_suspect'
decisions (id, invoice_id FK, agent_inputs JSONB, gate_weights JSONB, final_score, final_decision,
reasoning_text,
decided_at)
```

Prompt 2.2 — Groq client and RabbitMQ setup

```
Uncomment the RabbitMQ service in docker-compose.yml. In /ai-service, create:
- groqClient.js wrapping calls to the Groq API (GROQ_API_KEY, model llama-3.3-70b-versatile), with a
callGroqStructured(systemPrompt, userPrompt, jsonSchema) function that requests strict JSON and safely
parses it
- A RabbitMQ consumer listening on 'invoice.submitted' (update the backend's invoice upload route from Flow
1 to
publish to this queue after inserting the invoices row)
```

Prompt 2.3 — Agent 1: OCR & Structuring

```
Build POST /agents/ocr in /ai-service:
1. Accepts invoice_id, fetches the file path, runs Tesseract OCR (tesseract.js) for raw text
2. Sends the raw text to Groq to extract: vendor name, invoice number, line items, total amount, GST
amount,
GSTIN, invoice date — strict JSON matching a defined schema
3. Computes confidence_score from OCR word-confidence, or from how many expected fields were successfully
extracted
4. Inserts a document_extractions row with raw_ocr_text, structured_data, confidence_score, bounding_boxes
(empty
array for now — can be added later as a polish item)
```

Prompt 2.4 — Agent 2: Semantic Matching (GRN-based, not PO-based)

```
Build POST /agents/matching in /ai-service:
1. Fetches the invoice's structured_data and its LINKED goods_receipt_notes row via grn_id — compare
against
the GRN's received_quantity, NOT the PO's agreed_quantity directly. A partial delivery (invoice for 480
against a GRN that also says 480) must be a full match, even though the PO said 500.
2. Separately, informationally only, check the PO's cumulative_invoiced_quantity + this invoice against
agreed_quantity — only relevant for catching genuine overbilling across multiple invoices
3. Set issue_type: 'informational' for a legitimate partial delivery, 'mismatch' if quantities/amounts
genuinely
don't match the GRN
4. confidence_score reflects OCR read quality on matched line items; data_volume = number of comparable
line items
5. Insert a matching_results row
```

Prompt 2.5 — Agent 3: Compliance

```
Build POST /agents/compliance in /ai-service:
1. Validates GSTIN format via regex/checksum
2. Calls a GST verification function — real sandbox call if GSTN_SANDBOX_API_KEY is set, otherwise a
clearly
commented mock function
3. Independently recalculates the tax that SHOULD apply for the invoice date, using a small lookup table of
GST
rates by category — never compares against the PO's original stored tax estimate, which is only a
```

reference

4. Insert a `compliance_checks` row with `gst_valid`, `gst_details`, `issues_found`, `confidence_score`, `data_volume`, `gst_rate_used`, `gst_rate_reference_date`

Prompt 2.6 — Agent 8: The Decision Engine (Context Gate) — build this before Agents 4-7

Build `POST /agents/decide` in `/ai-service`. For now, this reads only Agents 2 and 3 (Matching, Compliance) — Fraud and Vendor Risk get added in Flow 3, and this route will be updated then to include them.

1. Fetches `matching_results` and `compliance_checks` for the invoice
2. Applies the Context Gate formula exactly:
$$\text{raw_weight_i} = \text{confidence_i} * \text{Math.log}(1 + \text{data_volume_i})$$
$$\text{gate_weight_i} = \text{raw_weight_i} / \text{sum}(\text{raw_weight_all_agents})$$
$$\text{final_score} = \text{sum}(\text{gate_weight_i} * \text{agent_verdict_score_i})$$
Convert each agent's verdict into a 0-1 numeric score before applying the formula.
3. `final_decision`: 'auto_approved' if `final_score` is above a configurable threshold, else 'flagged'
4. Generate `reasoning_text` via Groq — a plain-English paragraph naming which agent's signal drove the decision
(e.g. "flagged primarily due to Matching's low-confidence mismatch on a poorly scanned invoice")
5. Insert a `decisions` row with `agent_inputs`, `gate_weights`, `final_score`, `final_decision`, `reasoning_text`
6. Wire the RabbitMQ consumer from Prompt 2.2 to call OCR, then Matching + Compliance, then this route, in sequence

Prompt 2.7 — Frontend: Invoice Review page

Build `/frontend/company/invoice-review.html` (finance role): shows the invoice, a table of any mismatched fields, a confidence badge ("System confidence: High/Medium/Low" derived from the gate's aggregate confidence) next to the decision, the full `reasoning_text`, and an Approve Payment button. Add `GET /api/invoices/:id/review` in the backend returning the invoice plus its `document_extractions`, `matching_results`, `compliance_checks`, and latest `decisions` row.

Stop here and test again. Upload a clean invoice and confirm it auto-approves with sensible reasoning. Upload a deliberately mismatched invoice and confirm it flags correctly. Upload a partial-delivery scenario and confirm it does NOT get flagged. This is your core demo — make sure it's bulletproof before moving on.

FLOW 3 — Fraud Detection and Vendor Risk (completing the Decision Engine)

Milestone at the end: the Decision Engine now factors in fraud patterns and vendor track record, and a vendor's score updates automatically after every real transaction.

Prompt 3.1 — Vendor risk schema

Add to a new migration (`003_vendor_risk.sql`):

```
vendor_risk_scores (id, company_id FK, vendor_id FK, score, on_time_delivery_pct, dispute_rate,
  invoice_accuracy_pct, data_volume DEFAULT 0, data_source DEFAULT 'platform_verified',
  platform_verified_event_count DEFAULT 0, last_updated, UNIQUE(company_id, vendor_id))
fraud_flags (id, vendor_id FK, invoice_id FK nullable, flag_type, severity, evidence JSONB,
  confidence_score,
  resolved DEFAULT false)
```

Prompt 3.2 — Neo4j setup and Agent 4: Fraud Detection

- Uncomment the Neo4j service in `docker-compose.yml`. In `/ai-service`, build `POST /agents/fraud`:
1. Connect via `neo4j-driver` using `NEO4J_URI/NEO4J_USER/NEO4J_PASSWORD`
 2. Cypher query 1: does this vendor's `BankAccount` or `Address` share a `SHARES_BANK_ACCOUNT_WITH` or `SHARES_ADDRESS_WITH` relationship with a different Vendor node (shell company detection)
 3. Cypher query 2: multiple invoices against the same `PurchaseOrder`, each below the auto-approval threshold, submitted close together in time, WITHOUT proportional real `GoodsReceiptNote` backing (split-billing) — the distinguishing signal from a legitimate partial invoice, which always has real GRN backing
 4. On a match, insert a `fraud_flags` row with high `confidence_score` (graph exact-matches are highly

reliable)

5. Build a `syncToNeo4j` helper called from the Flow 1 backend routes whenever a vendor registers or an invoice/PO is created, keeping Vendor/Invoice/PurchaseOrder/BankAccount/Address nodes and relationships up to date

Prompt 3.3 — Agent 5: Vendor Risk Scoring (exact update logic)

Build three routes in `/ai-service` implementing this exact running-average logic:

POST `/agents/vendor-risk/on-grn-confirmed` — compares `received_date` to `agreed_delivery_date`, then:

`new_value = (old_value * old_data_volume + (onTime ? 100 : 0)) / (old_data_volume + 1)`

`new_data_volume = old_data_volume + 1`

`platform_verified_event_count += 1`

scoped to the `(company_id, vendor_id)` pair — create the row if it doesn't exist yet, with `data_volume=1` on first event.

POST `/agents/vendor-risk/on-dispute-resolved` — same formula applied to `dispute_rate`.

POST `/agents/vendor-risk/on-invoice-decision` — same formula applied to `invoice_accuracy_pct`, using whether `final_decision` was NOT 'flagged' as the event value.

Wire these into the Flow 1/2 routes: call `on-grn-confirmed` from the GRN route, `on-invoice-decision` from the end of Agent 8's decide route.

Prompt 3.4 — Update Agent 8 to include Fraud and Vendor Risk

Modify POST `/agents/decide` from Flow 2: now also fetch `fraud_flags` and `vendor_risk_scores` (scoped to this invoice's `company_id` and `vendor_id` — never any other company's row) for the invoice's vendor. Include both in the Context Gate formula alongside Matching and Compliance. If a high-severity `fraud_flags` row exists, set `final_decision='rejected'/'suspicious'` immediately, skipping the normal gate calculation, and route this to a Platform Admin fraud review queue rather than the normal flagged path.

Stop and test. Run a scenario with a planted fraud pattern (two vendors sharing a bank account, or a split invoice) and confirm it's caught. Confirm a vendor's score updates after a GRN is recorded — check the database directly to see `data_volume` and the score change.

FLOW 4 — Quotation Ranking and Vendor Communication (completing all 8 agents)

Milestone at the end: the Quotation Comparison page shows a real AI-ranked list with plain-English reasoning, and flagged invoices get an AI-drafted dispute message ready for Finance to review and send.

Prompt 4.1 — Agent 6: Quotation Ranking

Build POST `/agents/rank-quotations` in `/ai-service`:

1. Fetch all quotations for a requirement, plus each vendor's `vendor_risk_scores` row scoped to this requirement's `company_id` (never `vendor_platform_summary` — the gate must only use this company's own data)
 2. Compute three signals per quotation: price competitiveness, delivery fit, past performance (neutral midpoint with `confidence=0` if no `vendor_risk_scores` row exists)
 3. Apply the same Context Gate formula as Agent 8, with these three signals
 4. Generate `ai_rank_reasoning` via Groq, explicitly mentioning when a vendor's ranking leaned on price/delivery due to limited history
 5. Update each quotation's `ai_rank_score` and `ai_rank_reasoning`
- Update the backend's GET `/api/requirements/:id/quotations` route (from Flow 1) to call this instead of sorting by price directly.

Prompt 4.2 — Agent 7: Vendor Communication

Build POST `/agents/draft-dispute` in `/ai-service`: given an invoice's `matching_results/compliance_checks`, call

Grog to draft a short, polite, factual mismatch explanation (never accusatory), under 150 words. Insert a vendor_communications row with status='pending_send'. Call this automatically from Agent 8's decide route whenever final_decision='flagged' and no fraud_flags exist.

Prompt 4.3 — Backend: dispute routes

Add to /backend:

- GET /api/vendor-communications?status=pending_send (finance)
- PUT /api/vendor-communications/:id (finance) — edit draft_text
- POST /api/vendor-communications/:id/send (finance) — sets status='sent', sent_by, sent_at
- POST /api/vendor-communications/:id/resolve — sets resolved=true, calculates resolution_time_hours, calls the on-dispute-resolved route from Flow 3

Prompt 4.4 — Frontend: quotation comparison and dispute pages

Update /frontend/company/quotation-comparison.html to show ai_rank_score and a collapsible ai_rank_reasoning row per vendor, with an Accept button. Build /frontend/company/disputes.html (finance): list of pending_send messages with an editable textarea and Send button. Build /frontend/vendor/disputes.html: vendor's view of flagged disputes with the AI explanation and a response box.

Stop and test. All 8 agents now exist and are wired together. Walk through the full BrightBuild/SteelCorp-style example end to end, live, one clean case and one flagged case.

FLOW 5 — Multi-Company Isolation and Legacy Vendor Import

Milestone at the end: two different companies can use the platform with the same vendor without their data mixing, and a company can seed a known vendor's history at onboarding.

Prompt 5.1 — Legacy import route

Build POST /agents/vendor-risk/legacy-import (company_admin only) in /ai-service: accepts vendor_id, estimated_transaction_count, estimated_on_time_pct, optional dispute_rate/invoice_accuracy_pct, and notes. Only allowed if no vendor_risk_scores row exists yet for this (company_id, vendor_id) — return 409 otherwise. Halves estimated_transaction_count before storing as data_volume, sets data_source='self_reported', platform_verified_event_count=0. This becomes the normal starting point for Flow 3's update routes — no special casing needed there.

Prompt 5.2 — Platform-wide aggregate view

Write a migration adding a PostgreSQL view vendor_platform_summary: aggregates across all companies for a given vendor_id, using ONLY rows/events counted in platform_verified_event_count (never self_reported data) — sum total verified events, weighted average on_time_delivery_pct, count of distinct company_id. Add GET /api/vendors/:id/platform-summary returning this view's row for a vendor — used by the vendor's own dashboard and, informationally only, by the Quotation Comparison page for vendors a company has no history with.

Prompt 5.3 — Frontend: legacy import and trust score pages

Add an "Import Legacy History" button to /frontend/company/vendor-directory.html (company_admin only, shown only when the vendor has no existing score), opening a modal with the fields from Prompt 5.1. Update /frontend/vendor/trust-score.html to show the vendor_platform_summary aggregate with a short explanation.

Stop and test. Create two company accounts both working with the same vendor, confirm their vendor_risk_scores rows stay independent, and confirm the aggregate view only reflects verified (not self-reported) data.

FLOW 6 — Vendor Voice Notification (OmniDimension)

Milestone at the end: a vendor gets an automated call/SMS the moment their quotation is accepted. This is a strong, low-effort demo moment — good to have, but genuinely optional if time is short.

Prompt 6.1 — Notification service

```
In /backend, create notifyVendor.js with notifyQuotationAccepted(vendorId, poId): fetches the vendor's phone_number and preferred_notification_channel, calls the OmniDimension API (OMNIDIMENSION_API_KEY, OMNIDIMENSION_AGENT_ID, OMNIDIMENSION_OUTBOUND_NUMBER_ID) with a short informational message about the accepted quotation. Insert a row into a new outbound_notifications table logging vendor_id, reference_type, reference_id, channel, status, provider_message_id, sent_at. Wrap the whole call in try/catch — never let a notification failure affect the PO-generation transaction that calls it (Flow 1's quotation accept route).
```

Stop and test. Accept a quotation and confirm the vendor receives the call/message, and that the flow still completes normally even if you temporarily use an invalid phone number to test the failure path.

FLOW 7 — Admin Interface, Testing Data, and Evaluation

Milestone at the end: Platform Admin can review fraud flags and vendor verifications, and you have real accuracy numbers to put in your report.

Prompt 7.1 — Admin routes and pages

```
Build /backend routes: GET/POST for vendor verification queue (approve/reject), GET/POST for fraud_flags (resolve/escalate), GET for a monthly agent_overrides aggregation. Build /frontend/admin pages: dashboard.html, vendor-verification.html, fraud-flags.html, override-log.html, settings.html — matching the routes above.
```

Prompt 7.2 — Synthetic dataset and evaluation

```
Write seed-synthetic-data.js generating: 5 vendors, 10 requirements, 20 quotations, 15 POs, 15 GRNs (10 exact, 3 partial, 2 with discrepancy), 15 invoices (8 clean, 4 legitimate-variance, 2 genuine mismatch, 1 planted fraud pattern). Print the expected ground-truth outcome for each invoice.

Then write evaluate.js: runs the full pipeline against this data, compares actual vs expected final_decision, computes % correctly auto-approved and % correctly flagged. Re-run Agent 8 with the Context Gate replaced by equal weighting (1/4 per agent) as a baseline, and print a comparison table — this is your key ablation-study result for the final report.
```

FLOW 8 — Deployment

Milestone at the end: the whole thing is live at a public URL you can demo from any device.

Prompt 8.1 — Production Dockerfiles

```
Write production Dockerfiles for /backend and /ai-service (multi-stage, node:20-alpine, non-root user). Write an nginx-based Dockerfile for /frontend with an nginx.conf proxying /api/* to the backend service.
```

Prompt 8.2 — Render deployment


```
Write a render.yaml defining a Postgres service, a web service for /backend and /ai-service with all required environment variables referenced (not hardcoded), and a static site service for /frontend. Note as comments: create a Neo4j AuraDB free instance and a CloudAMQP free instance separately (Render doesn't host these natively), and set every environment variable in the Render dashboard before the first deploy.
```

Prompt 8.3 — Smoke test

```
Write smoke-test.js using node-fetch that, against the deployed URL, walks through: register a vendor, register a company, log in as both, post a requirement, submit and accept a quotation, submit a GRN, submit an invoice, poll until a decision appears — printing PASS/FAIL at each step. Run this after every deploy, and again right before your demo.
```

If time runs out, this is the honest cutoff order

Everything through **Flow 4** is your non-negotiable core — it's the whole story: registration → AI-ranked quotes → PO → delivery → AI-verified invoice → explainable decision → dispute → payment. If you have that fully working, polished, and demoed cleanly, you have a strong final project regardless of what's below.

- **Flow 5, 6** (multi-company isolation, voice notification) are genuinely impressive additions but not required for the core story to make sense.
- **Flow 7's admin pages** matter for completeness but the evaluation script inside it matters more than the pages — prioritize the numbers over the UI if squeezed.
- **Flow 8** (deployment) — even a flawless localhost demo is acceptable if deployment genuinely runs out of time; a working local demo beats a broken deployed one every time.